A decorative graphic on the left side of the slide, consisting of a black crosshair with a blue square in the top-left quadrant, a red square in the bottom-left quadrant, and a yellow square in the bottom-right quadrant.

Computer Programming II (CSE2035)

Lab #7

Object Oriented Programming Part 3

Prof. Jaeseung Choi
Dept. of Computer Science and Engineering
Sogang University

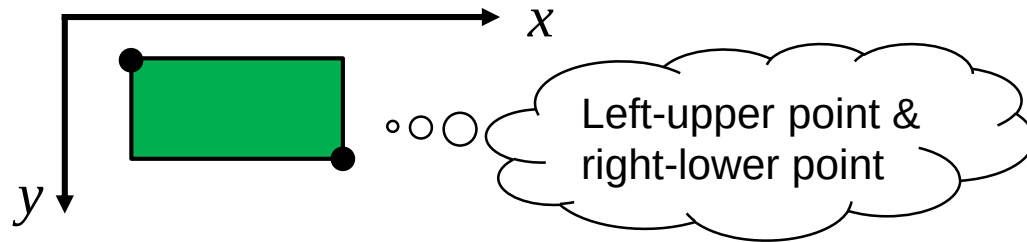
Homework 7: Abstract Class (20pt)

- Abstract class 'Shape' is defined in Shape.hpp
- This is a common interface for various 2D shapes (triangle, rectangle, circle, etc.)
 - We are promising that from now on, every class that represents a shape must include the following operations

```
class Shape {  
public:  
    virtual void Print() = 0;  
    virtual bool Shift(int delta_x, int delta_y) = 0;  
    virtual bool DoubleSize() = 0;  
    virtual bool Rotate() = 0;  
};
```

Part 1: Rectangle class

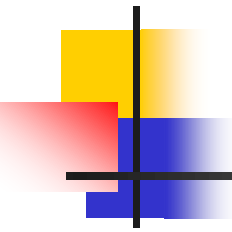
- Let's implement Rectangle class once again
- A rectangle can be represented with two points
 - Note that we don't need to maintain all the four points
 - Definition of Point class is provided in point.hpp



Part 1: Rectangle class

- **Rectangle class is defined by inheriting Shape class**
 - Properties, constructor, and Print() are given in rectangle.hpp
 - Remember that Print() is one of the virtual functions that we promised to define if we inherit Shape class

```
class Rectangle: public Shape {  
private:  
    Point left_up;  
    Point right_low;  
public:  
    Rectangle(int left_up_x, int left_up_y,  
              int right_low_x, int right_low_y) { ... }  
    void Print() { ... }
```



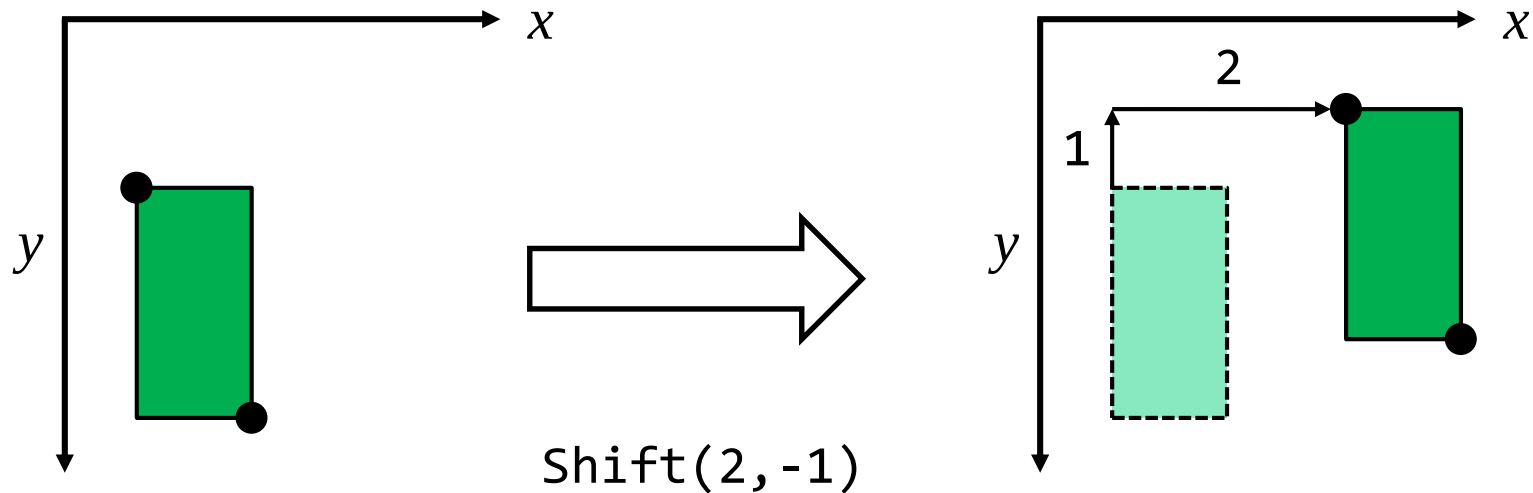
Part 1: Rectangle class

- **Implement the following functions in `rectangle.cpp`**
 - Detailed specifications are given in the comment of `rectangle.hpp` header file, so read them carefully

```
bool Shift(int delta_x, int delta_y);  
  
bool DoubleSize();  
  
bool Rotate();
```

Part 1: Rectangle class

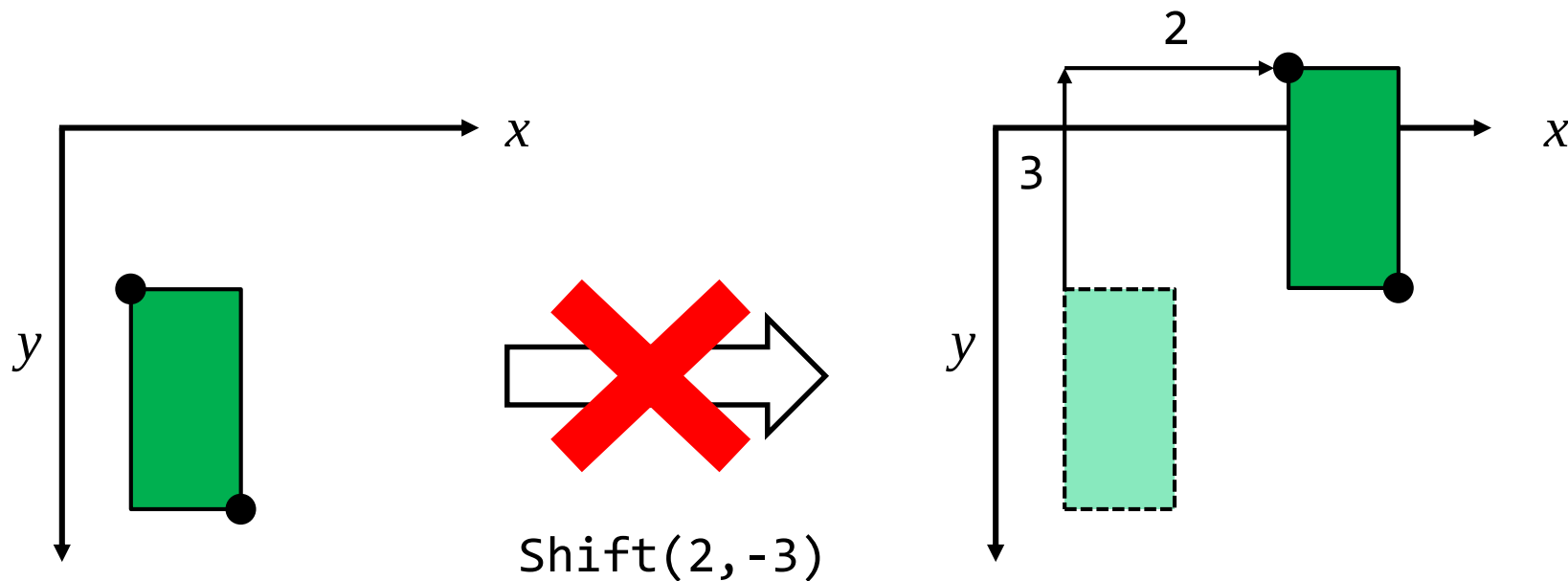
- Visual illustration of Shift()



Part 1: Rectangle class

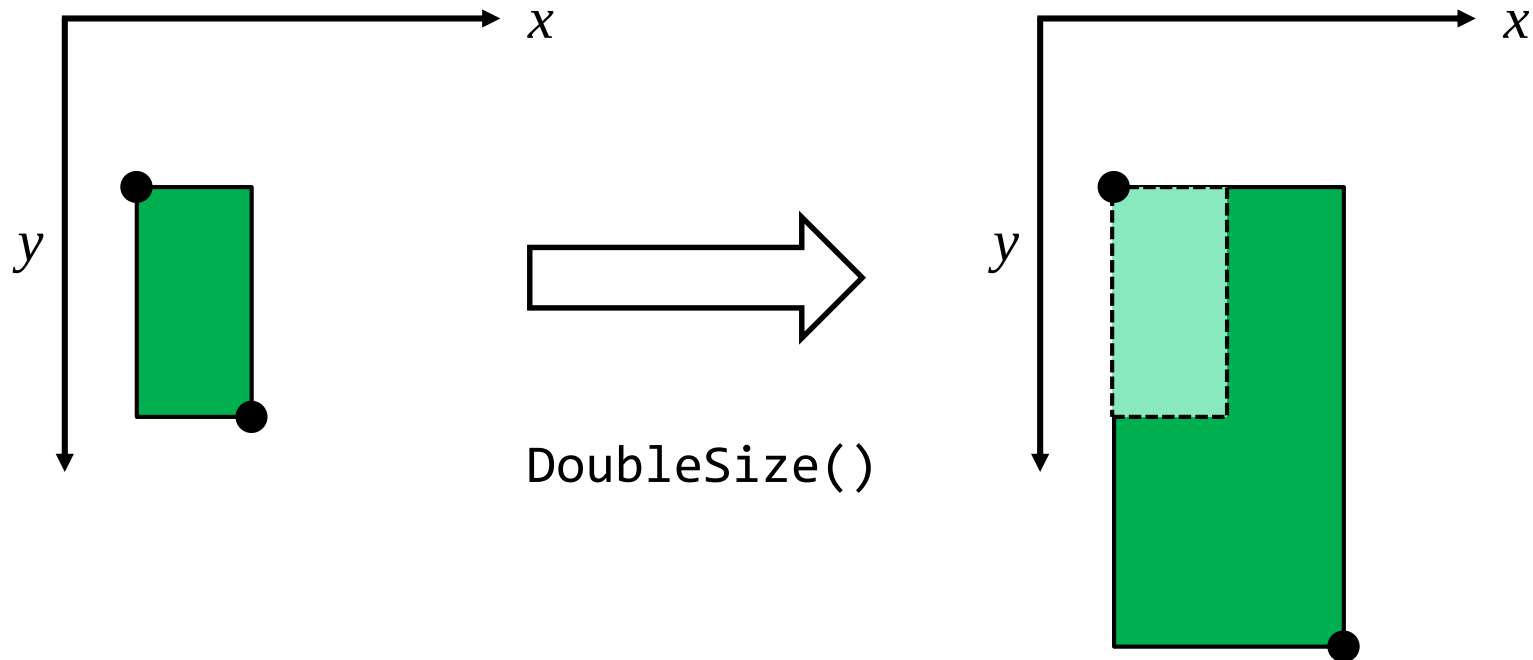
Visual illustration of invalid Shift()

- One of the four points has negative coordinate, so this operation must be rejected and return false



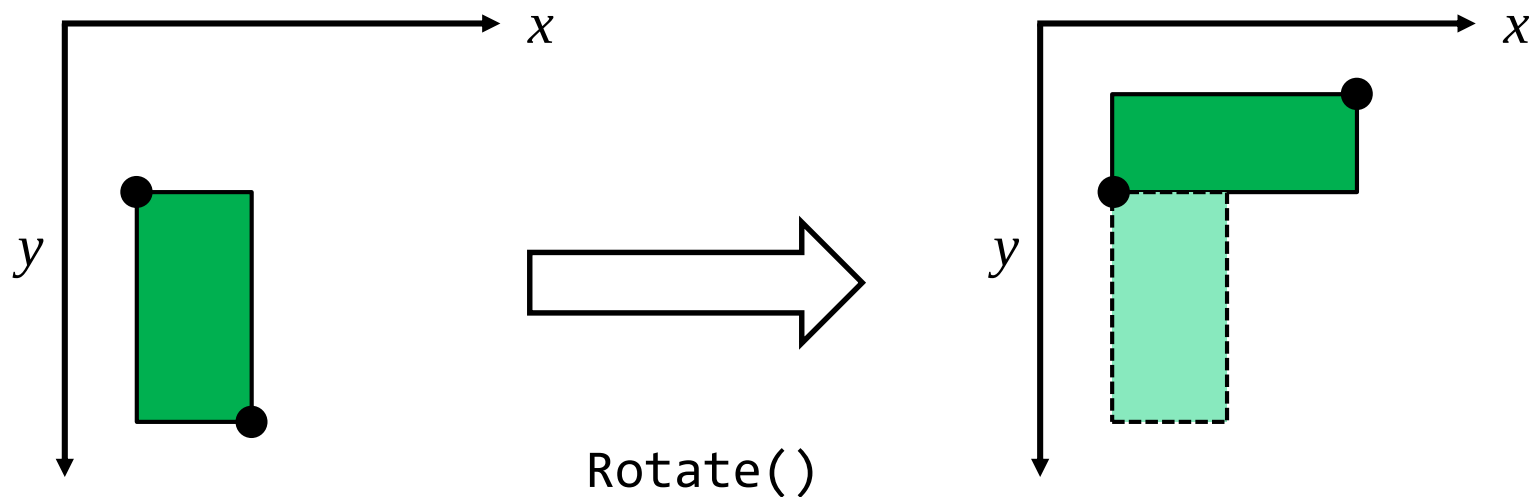
Part 1: Rectangle class

- Visual illustration of DoubleSize()



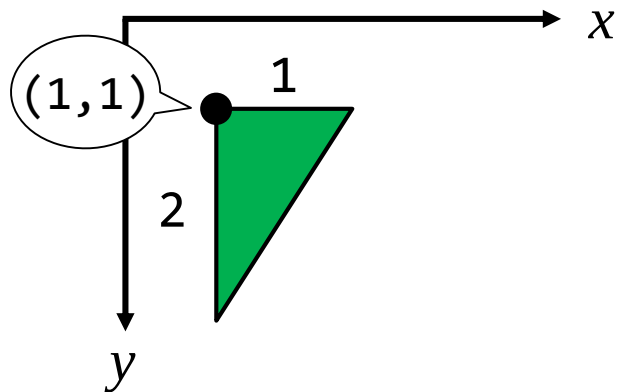
Part 1: Rectangle class

■ Visual illustration of Rotate()

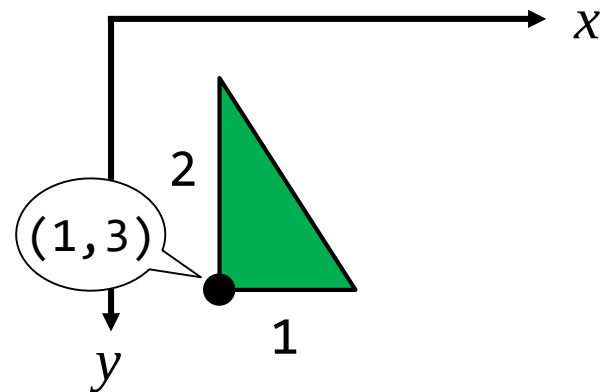


Part 2: RightTriangle class

- A right triangle can be uniquely defined by specifying
 - (1) the location of the point with right-angle
 - (2) relative position (offset) of the remaining two points



Point with 90°: (1,1)
 X-offset: +1
 Y-offset: +2

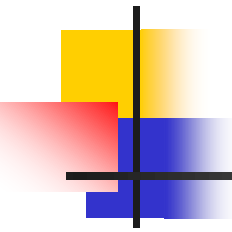


Point with 90°: (1,3)
 X-offset: +1
 Y-offset: -2

Part 2: RightTriangle class

- **RightTriangle class is defined by inheriting Shape class**
 - Properties, constructor, and Print() are given in right-triangle.hpp
 - Remember that Print() is one of the virtual functions that we promised to define if we inherit Shape class

```
class RightTriangle: public Shape {
private:
    Point ra_point;
    int x_offset;
    int y_offset;
public:
    RightTriangle(int ra_x, int ra_y, int x_off, int y_off)
    { ... }
    void Print() { ... }
```



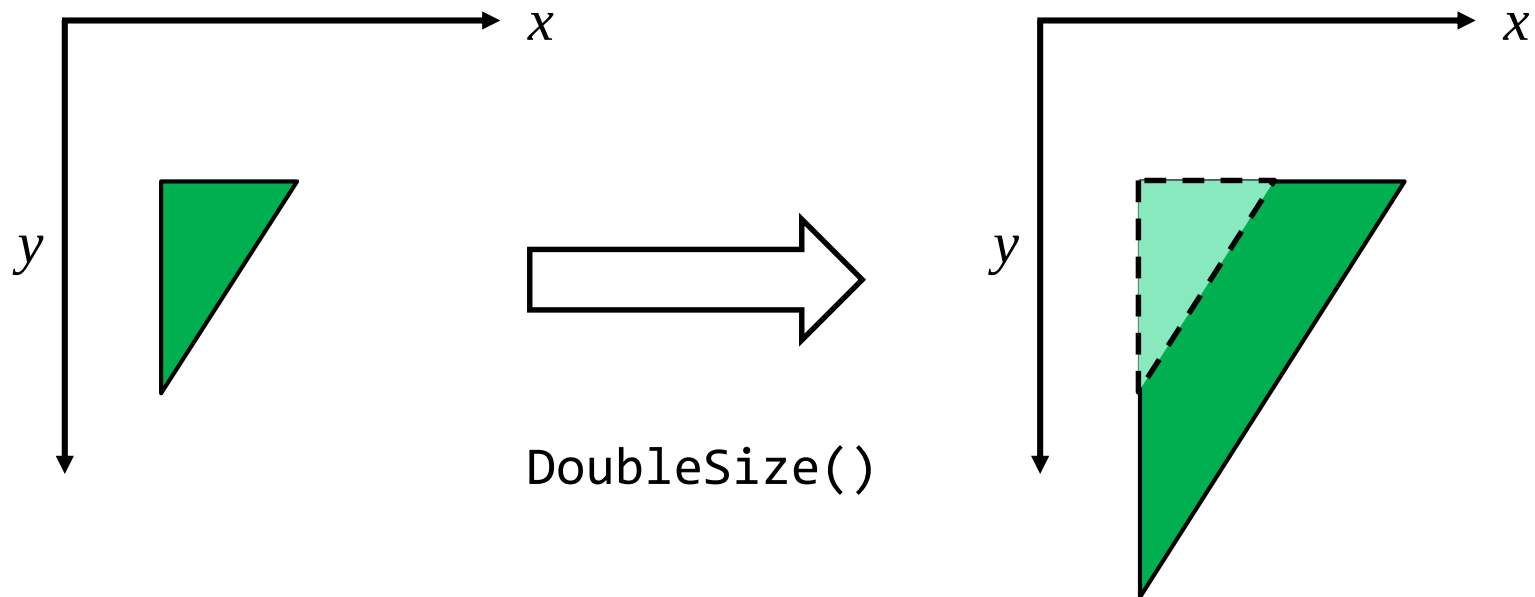
Part 2: RightTriangle class

- **Implement the following functions in right-triangle.cpp**
 - Detailed specifications are given in the comment of right-triangle.hpp header file, so read them carefully

```
bool Shift(int delta_x, int delta_y);  
  
bool DoubleSize();  
  
bool Rotate();
```

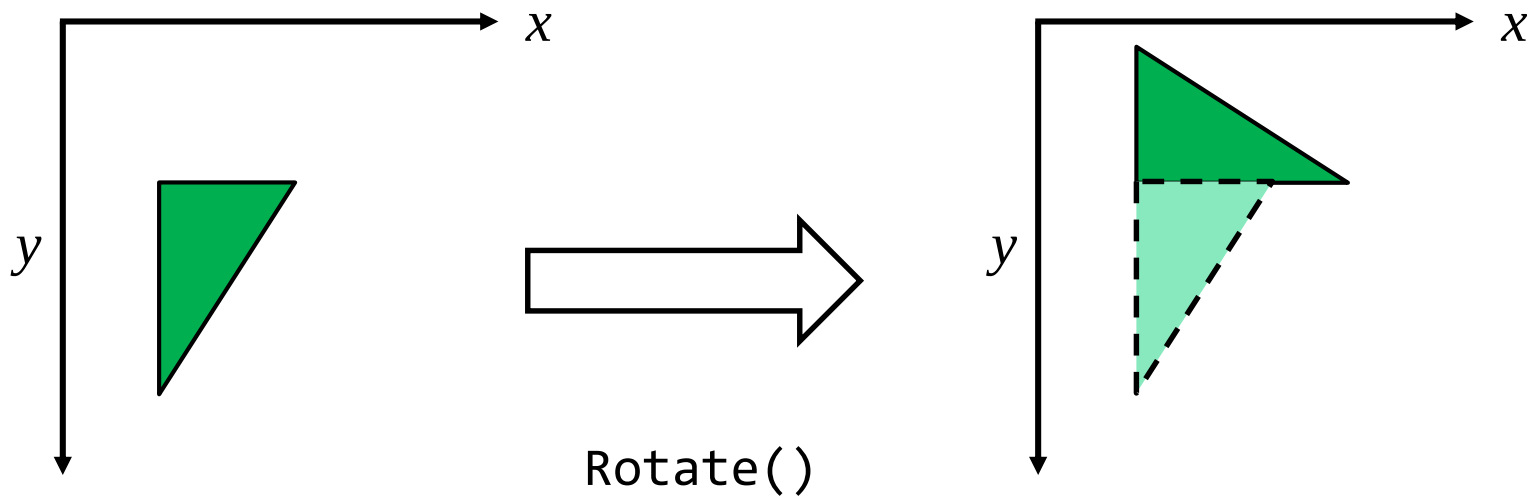
Part 2: RightTriangle class

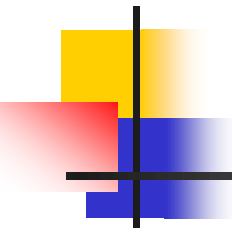
- Visual illustration of DoubleSize()



Part 2: RightTriangle class

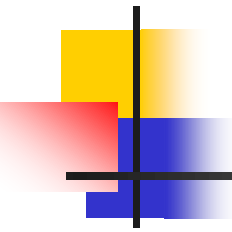
- Visual illustration of Rotate()



A decorative graphic consisting of overlapping yellow, red, and blue squares with a black crosshair.

Submission Guideline

- **Submit the completed `rectangle.cpp` and `right-triangle.cpp` files to “Lab #7 Assignment” in cyber campus**
 - You can fix `main.cpp` to test your classes, but don't submit it
 - Do not zip the directory, just upload these files directly
 - **Please do not change the name of files**
 - If the submitted file doesn't compile with the "make" command, cannot give you any point
- **Deadline is 11/11 (Fri) 23:59**
- **Late submission is allowed until 11/14 (Mon) 23:59**
 - 20% deduction

A decorative graphic consisting of overlapping yellow, red, and blue squares with a black crosshair.

Q & A Session
